

SHRINCS-Audit-Report

SHRINCS Solidity — Security Audit Report

Commit reviewed: `92edae2` — `92edae240cc6ad44b791b262533895a1b39f4470`

Executive summary

#	Severity	Finding
1	High — privileged precondition	Cross-policy stateful-leaf tracking desync re-opens consumed one-time keys
2	High — easy precondition	<code>RecoveryRotation</code> (disarmed) accepts untracked stateful one-time signatures
3	Medium — defense-in-depth	Stateless few-time budget reset on stateful-only rotation

Finding 1 — Cross-policy stateful-leaf tracking desync re-opens consumed one-time keys

Resolved: `9d862d6dbd69b145047b3080621f2daf58a19af6`

- **Location:** `ShrinCsAccountVerifierExample.sol` — `setStatefulPolicyMonotonicIndex` (:372), `setStatefulPolicyLeafBitmap` (:406), `precheckStatefulLeafUse` (:436), `commitStatefulLeafUse` (:504)

Description

The wrapper tracks spent one-time stateful leaves two different ways, in **disjoint storage that is never reconciled**: `MonotonicIndex` advances only the `nextStatefulLeafIndex` cursor; `LeafBitmap` sets only bits in `usedLeafBitmap[keyVersion]`. `commitStatefulLeafUse` writes to whichever one is active; neither setter imports the

other's state on a switch. The monotonic rollback guard `require(initialLeafIndex >= nextStatefulLeafIndex)` (:374) inspects only the cursor and is blind to bitmap consumption.

Attack trace

Consume leaves 1, 2, 3 under `MonotonicIndex` (cursor → 4, bitmap empty). Owner calls `setStatefulPolicyLeafBitmap()`. Now `precheckStatefulLeafUse(1)` returns `!isLeafUsed(1) = true` (:442) → leaf 1 re-accepted. The reverse is symmetric: bitmap-consuming leaf 100 leaves the cursor at 1, so `setStatefulPolicyMonotonicIndex(1)` passes `1 >= 1` (:374) and re-accepts it. Two signatures under one one-time WOTS-C leaf ⇒ Winternitz two-message reuse ⇒ any observer can forge further stateful signatures.

Finding 2 — **RecoveryRotation** (disarmed) accepts untracked stateful one-time signatures — now also via ERC-1271

Resolved: `544c43496fe032063a8df0252cc8f2d606fc0c3f`

- **Location:** `precheckStatefulLeafUse` (:436-443), `commitStatefulLeafUse` (:504), reached from `verifyStatefulAction` (:209) **and** the ERC-1271 chain `isValidSignature` (:91) → `decodeAndCheckStateful1271Envelope` (:123) → `isValidStatefulActionSignatureNow` (:451) → `precheckStatefulLeafUse` (:459)

Description

While `statefulPolicy == RecoveryRotation` and `recoveryMode == false`:

- `precheckStatefulLeafUse` only returns `false` for `RecoveryRotation && recoveryMode` (:438); with recovery **disarmed** it matches no branch and **falls through to** `return true` (:443).
- `commitStatefulLeafUse` handles only `MonotonicIndex` and `LeafBitmap` (:505, :510) — for `RecoveryRotation` it is a **no-op**; the leaf is never recorded.
- `verifyStatefulAction` has **no standing-policy gate** (compare `verifyStatelessAction:255`, which *does* block disarmed `RecoveryRotation`).

`setStatefulPolicyRecoveryRotation()` ends with `recoveryMode = false` (:397), so the wrapper **sits in exactly this sub-state by default** until `enterRecoveryMode()` is called.

Attack trace

Owner sets `RecoveryRotation` (the documented “lock to recovery-only” posture) without arming recovery. `verifyStatefulAction` for leaf 1 at nonce 5: precheck returns true, verification passes, commit no-ops, nonce advances. A second leaf-1 signature over nonce 6’s (different) message is accepted again → two WOTS-C signatures under one one-time leaf → forgery. Identically, an external caller of `isValidSignature(hash, 0x00 || envelope)` receives `MAGIC_VALUE` (0x1626ba7e) for the reusable leaf, since that path runs the same broken precheck.

Finding 3 — Stateless few-time budget reset on stateful-only rotation defeats

STATELESS_SIGNATURE_LIMIT

Resolved: 7a3aa19fda7c16b266a06cb7e0d158c20249378a

- **Location:** `rotateToFreshKey` (:288) → `SHRINCS.rotateStatefulViaStateless` (`SHRINCS.sol:70` ; rebuilds commitment from `nextStatefulKey.statefulPublicKey`, `currentPublicKey.pkSeed`, `currentPublicKey.hypertreeRoot` at `SHRINCS.sol:95-97` , i.e. **preserves the stateless key**) → `installFreshKey` (:537, unconditional `statelessSignaturesUsed = 0` at :546)

Description

The stateless FORS key is *few-time*, capped at `STATELESS_SIGNATURE_LIMIT = 220` via `statelessSignaturesUsed`. `135afff` correctly routed `rotateToFreshKey` through `rotateStatefulViaStateless`, which preserves the stateless key (only the stateful subkey changes). But `installFreshKey` still resets the counter to `0` on **every** rotation. Because the counter is effectively keyed to the bundle commitment (which changes each rotation) rather than to the stateless key material (unchanged), each stateful-only rotation grants the **same** few-time FORS key a fresh 2^{20} budget, resetting it.

Attack trace

`statelessSignaturesUsed = 1,048,575` for stateless key material (`pkSeed`, `hypertreeRoot`) → owner performs a stateful-only recovery rotation via `rotateToFreshKey` → `rotateStatefulViaStateless` preserves (`pkSeed`, `hypertreeRoot`) and changes only the

stateful subkey → the bundle commitment changes, so `installFreshKey` treats it as a fresh epoch and resets `statelessSignaturesUsed = 0` → the same few-time FORS/stateless key now receives another full `220` budget. The stateless recovery signature used to authorize the rotation is also not counted. Repeating this `N` times gives approximately `N · 220 + N` stateless uses under the same FORS key material, while the contract never observes the per-epoch counter exceeding `STATELESS_SIGNATURE_LIMIT`.